

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»



Инженерная школа информационных технологий и робототехники

Отделение информационных технологий

09.04.01 «Информатика и вычислительная техника»

Отчет по лабораторной работе №4
«Реализация глубокого сверточного автоэнкодера для шумоподавления»

МЕТОДЫ РАСПОЗНАВАНИЯ ОБРАЗОВ

Исполнитель:

студент группы

8BM22

Ямкин Н.Н.

Руководитель:

Доцент (ОИТ, ИШИТР)

Друки А.А.

Томск – 2023

Содержание

1. Цель работы	3
2. Задачи	3
3. Ход работы.....	5
4. Заключение	8

1. Цель работы

Реализовать глубокий сверточный автоэнкодер для шумоподавления на изображениях MNIST.

2. Задачи

1. Предобработать обучающие данные;
2. Добавить случайный шум к каждому изображению в массиве;
3. Реализовать архитектуру автоэнкодера;
4. Протестировать подавление шума на изображениях с помощью автоэнкодера.

Autoencoder (автокодер, автоэнкодер, АЕ) – нейронная сеть, которая копирует входные данные на выход. По архитектуре похож на персептрон. Автоэнкодеры сжимают входные данные для представления их в latent-space (скрытое пространство), а затем восстанавливают из этого представления output (выходные данные). Цель – получить на выходном слое отклик, наиболее близкий к входному.

Отличительная особенность автоэнкодеров — количество нейронов на входе и на выходе совпадает.

Автокодер состоит из двух частей:

1. **Энкодер:** отвечает за сжатие входа в latent-space.
2. **Декодер:** предназначен для восстановления ввода из latent-space.

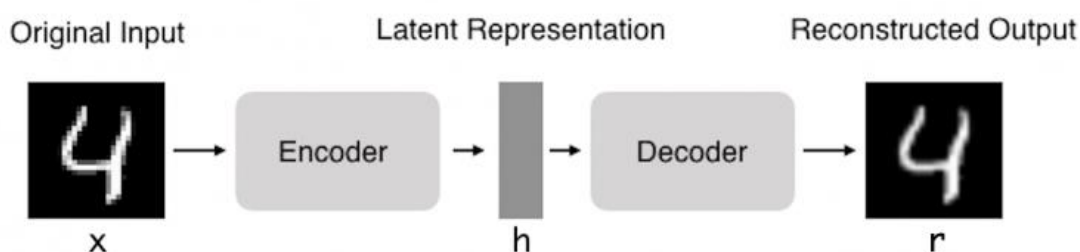


Рисунок 1 – Архитектура автоэнкодера

Если бы единственной задачей автоэнкодеров было копирование входных данных на выход, они были бы бесполезны. Исследователи рассчитывают на то, что скрытое представление h будет обладать полезными свойствами.

Этого добиваются созданием ограничений для задачи копирования. Один из способов получить полезные результаты от автоэнкодера – ограничить h размерами меньшими, чем x . В этом случае автокодирование называется неполным.

Автоэнкодер может начать выполнять задачу копирования, не извлекая полезной информации о распределении данных, если:

1. Размерность скрытого представления совпадает с размерностью входа;
2. Размерность скрытого представления больше, чем размерность входа;
3. Автоэнкодеру предоставляется слишком большой объем данных.

В этих случаях даже линейный кодер и линейный декодер копируют входные данные на выход, не изучая ничего полезного о распределении.

В идеале можно организовать любую архитектуру автокодера, задавая размер кода и емкость энкодера и декодера на основе сложности моделируемого распределения.

3. Ход работы

Создадим новый блокнот в Google Colab и импортируем необходимые для работы с изображениями библиотеки:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
```

Функции нормализации массива и добавления шума.

```
def preprocess(array):
    """
    Нормализует массив и преобразует его в соответствующий формат.
    """
    array = array.astype("float32") / 255.0
    array = np.reshape(array, (len(array), 28, 28, 1))
    return array

def noise(array):
    """
    Добавляем случайный шум к каждому изображению в массиве.
    """

    noise_factor = 0.9
    noisy_array = array + noise_factor * np.random.normal(
        loc=0.0, scale=1.0, size=array.shape
    )

    return np.clip(noisy_array, 0.0, 1.0)

def display(array1, array2):
    """
    Отобразим десять случайных изображений из массива
    """

    n = 10

    indices = np.random.randint(len(array1), size=n)
    images1 = array1[indices, :]
    images2 = array2[indices, :]

    plt.figure(figsize=(20, 4))
    for i, (image1, image2) in enumerate(zip(images1, images2)):
        ax = plt.subplot(2, n, i + 1)
```

```
plt.imshow(image1.reshape(28, 28))
plt.gray()
ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)

ax = plt.subplot(2, n, i + 1 + n)
plt.imshow(image2[:, :, 0])
plt.gray()
ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)

plt.show()
```

Загрузка набора данных MNIST и выполнение соответствующих операций по предварительной обработке и добавлению шума.

```
# Так как для кодирования и декодирования нам нужны только изображения из
набора данных, мы не будем использовать метки
(train_data, _), (test_data, _) = mnist.load_data()

# Нормализация изображений
train_data = preprocess(train_data)
test_data = preprocess(test_data)

# Создадим копию данных с добавленным шумом
noisy_train_data = noise(train_data)
noisy_test_data = noise(test_data)

# Отобразим данные и их версии с добавленным шумом
display(train_data, noisy_train_data)
```



Рисунок 2 – Исходные данные с добавленным шумом

Архитектура нейронной сети.

#Архитектура нейронной сети:

```
input = layers.Input(shape=(28, 28, 1))
```

Энкодер

```
x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(input)
```

```
x = layers.MaxPooling2D((2, 2), padding="same")(x)
```

```
x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(x)
```

```
x = layers.MaxPooling2D((2, 2), padding="same")(x)
```

Декодер

```
x = layers.UpSampling2D(size=(2, 2))(x)
```

```
x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(x)
```

```
x = layers.UpSampling2D(size=(2, 2))(x)
```

```
x = layers.Conv2D(64, (3, 3), activation="relu", padding="same")(x)
```

output layer

```
decoded = layers.Conv2D(1, (3, 3), activation="sigmoid",
```

```
padding="same")(x)
```

Модель автоэнкодера

```
autoencoder = Model(input, x)
```

```
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
```

```
autoencoder.summary()
```

```
Model: "model_1"
```

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 28, 28, 1)]	0
conv2d_5 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_6 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0
up_sampling2d_2 (UpSampling2D)	(None, 14, 14, 64)	0
conv2d_7 (Conv2D)	(None, 14, 14, 64)	36928
up_sampling2d_3 (UpSampling2D)	(None, 28, 28, 64)	0
conv2d_8 (Conv2D)	(None, 28, 28, 64)	36928

```
=====  
Total params: 92672 (362.00 KB)  
Trainable params: 92672 (362.00 KB)  
Non-trainable params: 0 (0.00 Byte)
```

Рисунок 3 – Архитектура нейронной сети

Обучим автоэнкодер.

```
# Обучим автоэнкодер, используя зашумленные изображения и чистые
изображения в качестве целевых. Необходимо, чтобы автоэнкодер научился
подавлять шум.
autoencoder.fit(
    x=noisy_train_data,
    y=train_data,
    epochs=50,
    batch_size=128,
    shuffle=True,
    validation_data=(noisy_test_data, test_data),
)
```

Сделаем прогноз на основе зашумленных данных и отобразим результаты автоэнкодера. Автоэнкодер должен подавлять шум.

```
# Теперь сделаем прогноз на основе зашумленных данных и отобразим
результаты автоэнкодера. Автоэнкодер должен подавлять шум.
predictions = autoencoder.predict(noisy_test_data)
display(noisy_test_data, predictions)
```

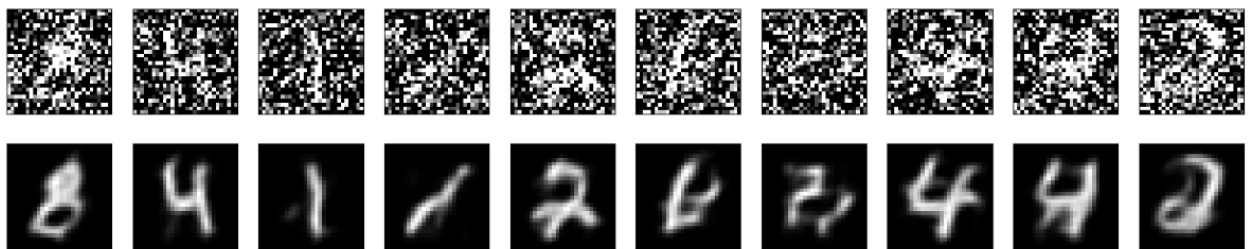


Рисунок 4 – Результат работы автоэнкодера

4. Заключение

В результате выполнения данной лабораторной работы произошло ознакомление с глубоким конволюционным автоэнкодером (DCA) для обработки набора данных MNIST.